

Marina One

Sistema SaaS de Gestão Integrada de Marina

Multi-Tenant · PostgreSQL · Node.js 20

MANUAL DE IMPLANTAÇÃO E ARQUITETURA

Versão 2.0 — 2025

Sumário

1	Visão Geral do Sistema v2.0	pág. 3
2	Arquitetura Multi-Tenant	pág. 4
3	Requisitos de Infraestrutura	pág. 6
4	Licenças e Dependências	pág. 7
5	Implantação Passo a Passo	pág. 8
6	Gerenciamento de Tenants (Super-Admin)	pág. 12
7	Desenvolvimento Local com Docker	pág. 13
8	Checklist Pré-Entrega	pág. 14
9	Custos de Operação	pág. 15
10	Processo de Atualização	pág. 16
11	Escalabilidade e Roadmap	pág. 19
12	Suporte e Contato	pág. 20

1 Visão Geral do Sistema v2.0

O **Marina One v2.0** é um sistema **SaaS multi-tenant** de gestão integrada para marinas. A versão 2.0 migrou completamente a arquitetura para **PostgreSQL** com isolamento por schema (schema-per-tenant), permitindo atender múltiplas marinas em uma única instância do sistema com segurança e escalabilidade.

Stack tecnológico v2.0

Node.js v20 LTS	postgres.js 3.x	PostgreSQL 16	Chart.js · HTML5 / JS
-----------------	-----------------	---------------	-----------------------

Módulos do sistema

Dashboard	Fila de Operações	Clientes
Embarcações	Vagas	Contratos
Financeiro	Loja / PDV	Manutenção
Analytics	Alertas	Configurações

Principais mudanças da v1.x para v2.0

Aspecto	v1.x (SQLite)	v2.0 (PostgreSQL SaaS)
Banco de dados	SQLite — arquivo local marina.db	PostgreSQL — schema por tenant
Multi-tenant	Instância por cliente	Compartilhada — schema isolado
Atualização	Servidor por servidor	Um deploy atualiza todos
Escalabilidade	~10 usuários simultâneos	Centenas de tenants
Deploy	VPS ou servidor local	Railway, Render ou VPS + Docker
Super-admin	Não existia	Painel /api/superadmin/*

2 Arquitetura Multi-Tenant

Modelo: Schema-per-Tenant no PostgreSQL

Cada marina possui um **schema PostgreSQL exclusivo** (`marina_<slug>`). As tabelas de uma marina jamais são acessíveis por outra — o isolamento é garantido pelo banco de dados. Um schema global (`saas`) armazena o catálogo de tenants e os super-admins.

```
Banco de Dados PostgreSQL

saas.tenants — registro de todas as marinas
saas.super_admins — acesso de Arthur ao painel SaaS

marina_porto-belo.* — schema exclusivo da Marina Porto Belo
marina_angra.* — schema exclusivo da Marina Angra
marina_buzios.* — schema exclusivo da Marina Búzios
...
```

Fluxo de Resolução de Tenant por Requisição

1	Requisição HTTP chega	GET /api/clients — com header X-Tenant-Slug: porto-belo
2	Middleware de tenant	Valida slug contra saas.tenants, verifica se está ativo
3	Middleware de auth	Valida JWT — verifica tenant_slug do token vs requisição
4	Pool de conexão	getTenantPool('porto-belo') — search_path = marina_porto-belo
5	Route handler	SELECT * FROM clients — busca apenas no schema porto-belo
6	Resposta	Dados isolados retornados ao cliente

JWT Vinculado ao Tenant

Cada token JWT inclui `tenant_slug`. O middleware de autenticação rejeita tokens de tenants diferentes, impedindo que um usuário da Marina Porto Belo acesse dados da Marina Angra mesmo com token válido.

```
{
  "user_id": 42,
  "email": "gerente@portobelo.com",
  "role": "admin",
  "tenant_id": 3,
  "tenant_slug": "porto-belo",
  "exp": 1750000000
}
```

Resolução de Tenant — Estratégias (em prioridade)

1º Prioridade	Header X-Tenant-Slug (injetado pelo NGINX — produção)
---------------	---

2º Prioridade	Variável SINGLE_TENANT_SLUG no ambiente (modo single-tenant)
3º Prioridade	Subdomínio do Host: porto-belo.marinaone.com.br
4º Prioridade	Query string ?tenant=porto-belo (apenas em desenvolvimento)

3 Requisitos de Infraestrutura

Opção A — SaaS Gerenciado (Recomendado)

Melhor custo-benefício para SaaS: zero ops, escalabilidade automática, deploy em segundos via git push.

Banco de dados	Neon.tech (PostgreSQL serverless) — plano gratuito até 0,5 GB, ~R\$ 30/mês para produção
Aplicação	Railway.app ou Render.com — plano Hobby R\$ 25–60/mês · zero configuração
Domínio	R\$ 40–60/ano · wildcard *.marinaone.com.br via Cloudflare (SSL gratuito)
Total estimado	R\$ 55–100/mês para múltiplas marinas — custo fixo independente do número de tenants

Opção B — VPS com PostgreSQL (Controle Total)

Servidor	2 vCPU, 4GB RAM, 40GB SSD — Hostinger VPS, DigitalOcean, Vultr
Custo mensal	R\$ 80–150/mês
PostgreSQL	Instalado no próprio servidor ou serviço gerenciado (Supabase, Neon)
SSL	Certbot wildcard via Cloudflare DNS-01 + NGINX
Backup	pg_dump diário via cron · retenção 30 dias

Opção C — Servidor Local (Single-Tenant, sem escala)

Hardware	Mini PC (Intel NUC) 8GB RAM — R\$ 800–1.500 (custo único)
PostgreSQL	Instalado localmente via Docker: docker compose up -d
Acesso	Tailscale ou Cloudflare Tunnel (gratuito)
Modo	SINGLE_TENANT_SLUG no .env — sem subdomínios, uma marina por instância

4 Licenças e Dependências

Componente	Versão	Licença	Custo
Node.js	v20 LTS	MIT	Gratuito
postgres.js	3.x	MIT	Gratuito
PostgreSQL	16	PostgreSQL (BSD)	Gratuito
Chart.js (CDN)	4.x	MIT	Gratuito
Ubuntu / Alpine	22.04 LTS	Free	Gratuito
Docker	24+	Apache 2	Gratuito
TOTAL	—	—	R\$ 0

Stack 100% open source. Nenhuma dependência paga — o custo de operação é exclusivamente de infraestrutura (hospedagem e domínio).

5 Implantação Passo a Passo

5A — Deploy SaaS Gerenciado (Railway + Neon) — Recomendado

A forma mais rápida de colocar o sistema no ar para múltiplas marinas. O Railway detecta o Dockerfile e sobe automaticamente.

1	Neon.tech	Criar banco PostgreSQL gratuito · copiar a DATABASE_URL
2	Cloudflare	Registrar domínio · configurar wildcard *.marinaone.com.br
3	GitHub	Criar repositório privado · push do código
4	Railway.app	New Project → Deploy from GitHub · variáveis de ambiente abaixo
5	Health check	Acessar /api/version e confirmar resposta com version: 2.0.0
6	1ª Marina	POST /api/superadmin/tenants para criar o primeiro tenant

Variáveis de ambiente obrigatórias no Railway:

```
DATABASE_URL=postgres://usuario:senha@ep-xxx.neon.tech/marinaone?sslmode=require
DB_SSL=true
JWT_SECRET=string_aleatoria_minimo_32_chars
BASE_DOMAIN=marinaone.com.br
SUPER_ADMIN_EMAIL=arthur@marinaone.com.br
SUPER_ADMIN_PASSWORD=senha_super_admin_forte
NODE_ENV=production
```


5B — Deploy em VPS com PostgreSQL

Passo 1 — Preparar servidor Ubuntu 22.04

```
sudo apt update && sudo apt upgrade -y
# Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs git nginx certbot python3-certbot-nginx
node -v # Verificar: v20.x.x

# PostgreSQL 16
sudo apt install -y postgresql-16
sudo -u postgres psql -c "CREATE USER marinaone WITH PASSWORD 'senha_forte';"
sudo -u postgres psql -c "CREATE DATABASE marinaone OWNER marinaone;"
```

Passo 2 — Clonar e configurar a aplicação

```
git clone https://github.com/seuusuario/marina-one.git
cd marina-one
npm install --omit=dev

# Criar .env
cp .env.example .env
nano .env # Preencher DATABASE_URL, JWT_SECRET, etc.
```

Passo 3 — PM2 (processo sempre online)

```
sudo npm install -g pm2
pm2 start server.js --name marina-one
pm2 startup && pm2 save
# Aguardar startup — o sistema cria os schemas automaticamente
```

Passo 4 — NGINX com wildcard + HTTPS

Copie o arquivo `nginx/nginx.conf` do projeto para `/etc/nginx/sites-available/marina-one` e habilite-o:

```
# Habilitar site
sudo ln -s /etc/nginx/sites-available/marina-one /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

# Wildcard SSL via Cloudflare DNS-01
sudo certbot certonly --dns-cloudflare \
  --dns-cloudflare-credentials ~/.cloudflare.ini \
  -d marinaone.com.br -d '*.marinaone.com.br'
```

Passo 5 — Backup automático do banco

```
# /etc/cron.d/marina-backup
0 2 * * * postgres pg_dump marinaone \
  | gzip > /backups/marina-$(date +%Y-%m-%d).sql.gz
0 3 * * * root find /backups -name 'marina-*.gz' -mtime +30 -delete
```

5C — Modo Single-Tenant (uma marina por instância)

Para implantar o sistema para uma única marina sem subdomínios, defina a variável `SINGLE_TENANT_SLUG` no ambiente. O sistema cria e migra o schema automaticamente no primeiro start:

```
# .env - modo single-tenant
DATABASE_URL=postgres://marinaone:senha@localhost:5432/marinaone
JWT_SECRET=sua_chave_secreta_aqui
SINGLE_TENANT_SLUG=porto-belo
SINGLE_TENANT_NAME=Marina Porto Belo
ADMIN_EMAIL=admin@portobelo.com
ADMIN_PASSWORD=senha_inicial_admin
```

Vantagem do modo single-tenant: funciona exatamente como a v1.x do ponto de vista do usuário final — mesma URL, sem subdomínio. O frontend **não precisa de nenhuma alteração**. Ideal para a primeira comercialização.

Passo 6 — Personalização do tenant

- Trocar credenciais do admin em Usuários → admin@marina.com / marina123
- Personalizar nome da marina em Configurações → Marina
- Criar usuários para a equipe (Operador, Loja/PDV)
- Configurar horários e checklists de operação em Configurações
- Cadastrar vagas secas e molhadas na tela de Vagas

6 Gerenciamento de Tenants — Painel Super-Admin

O painel super-admin é exclusivo do Arthur Noli (SaaS Owner). Permite criar novas marinas, consultar estatísticas e ativar/desativar tenants — tudo via API REST autenticada separadamente do sistema das marinas.

Autenticação do Super-Admin

```
POST /api/superadmin/auth/login
{
  "email": "arthur@marinaone.com.br",
  "password": "SaaS@Marina2025!"
}

# Resposta → { "token": "eyJ...", "admin": { "name": "Arthur Noli" } }
```

Endpoints disponíveis

Método	Endpoint	Função
POST	/api/superadmin/auth/login	Autenticação do super-admin
GET	/api/superadmin/tenants	Listar todas as marinas
POST	/api/superadmin/tenants	Criar e provisionar nova marina
PUT	/api/superadmin/tenants/:slug	Atualizar nome, plano ou status
GET	/api/superadmin/tenants/:slug/stats	KPIs da marina (clientes, receita...)

Criar uma nova marina (provisionamento completo)

```
POST /api/superadmin/tenants
Authorization: Bearer <super-admin-token>
{
  "slug": "porto-belo",
  "name": "Marina Porto Belo",
  "plan": "professional",
  "adminEmail": "admin@portobelo.com",
  "adminPassword": "senha_segura_123",
  "adminName": "Gerente Porto Belo"
}

# O sistema automaticamente:
# 1. Registra em saas.tenants
# 2. Cria schema marina_porto-belo
# 3. Executa todas as migrações SQL
# 4. Faz seed do admin, operador e permissões
# Marina está pronta em segundos.
```

7 Desenvolvimento Local com Docker

O projeto inclui um `docker-compose.yml` completo para subir o ambiente de desenvolvimento em um único comando, com PostgreSQL 16.

```
# Subir tudo (PostgreSQL + App)
docker compose up -d

# Verificar se o app está rodando
curl http://localhost:3000/api/version

# Ver logs em tempo real
docker compose logs -f app

# Parar o ambiente
docker compose down
```

O Docker Compose configura automaticamente a variável `SINGLE_TENANT_SLUG=demo`, então o sistema sobe com a marina **demo** pronta para uso imediato.

Sem Docker — Node.js + PostgreSQL local

```
# 1. PostgreSQL local instalado e rodando
# 2. Criar .env com DATABASE_URL
cp .env.example .env # editar DATABASE_URL

# 3. Instalar dependências
npm install

# 4. Iniciar (schemas criados automaticamente no boot)
npm start
```

8 Checklist Pré-Entrega

Infraestrutura e Servidor

- DATABASE_URL configurada e banco PostgreSQL acessível
- JWT_SECRET com mínimo 32 caracteres aleatórios (não use o padrão do .env.example)
- HTTPS funcionando — cadeado verde em todos os acessos
- Wildcard SSL *.marinaone.com.br configurado (ou domínio do cliente)
- Endpoint /api/version respondendo com version: 2.0.0
- Firewall configurado (apenas portas 80, 443 e 22 abertas)
- Backup automático do banco testado e verificado
- PM2 iniciando no boot: reiniciar servidor e confirmar

Configuração do Tenant

- Credenciais do super-admin trocadas (não usar senha padrão em produção)
- Tenant da marina criado via POST /api/superadmin/tenants
- Admin da marina com senha trocada (padrão: marina123)
- Nome e dados da marina configurados em Configurações
- Usuários da equipe criados (Operador, Loja/PDV)
- Vagas secas e molhadas cadastradas
- PIX configurado em Loja → Configurações PIX
- Horários de operação configurados

Testes e Qualidade

- Testar login com todos os perfis (admin, operador, loja, cliente)
- Testar fluxo completo: criar cliente → embarcação → contrato → cobrança

-
- Testar fila de operações (descida, subida, atracação)
-
- Testar PDV — criar pedido, confirmar pagamento, delivery
-
- Verificar responsividade em celular
-
- Demonstração completa com o cliente
-

9 Custos de Operação

Modelo SaaS — Custo Fixo para N Marinas

Com a arquitetura multi-tenant, o custo de infraestrutura é **fixo** independente do número de marinas. Adicionar a 10ª marina tem o mesmo custo da 1ª — apenas armazenamento aumenta marginalmente.

Item	Serviço	Custo
Banco de dados PostgreSQL	Neon.tech Pro	~R\$ 30/mês
Aplicação (hospedagem)	Railway / Render	R\$ 25–60/mês
Domínio wildcard	Cloudflare + registro	~R\$ 5/mês (pró-rata)
SSL wildcard	Cloudflare (gratuito)	R\$ 0
Licenças de software	—	R\$ 0
TOTAL (SaaS gerenciado)	—	R\$ 60–95/mês

Projeção de Receita (exemplo com 5 marinas)

Marina	Plano	Mensalidade
Marina Porto Belo	Pro	R\$ 1.000/mês
Marina Angra	Pro	R\$ 1.000/mês
Marina Búzios	Starter	R\$ 700/mês
Marina Paraty	Pro	R\$ 1.000/mês
Marina Cabo Frio	Starter	R\$ 700/mês
RECEITA TOTAL	—	R\$ 4.400/mês
Custo infraestrutura	—	R\$ 80/mês
MARGEM	—	R\$ 4.320/mês (98%)

10

Processo de Atualização

A maior vantagem do SaaS: atualiza todos de uma vez

Com a arquitetura multi-tenant, um único deploy atualiza **TODAS** as marinas **simultaneamente**. Não é mais necessário acessar cada servidor de cliente — basta fazer um git push ou acionar o deploy no Railway/Render.

Fluxo de atualização SaaS (Railway/Render)

```
# Desenvolvedor faz commit e push
git add . && git commit -m "feat: novo módulo de relatórios"
git push origin main

# Railway/Render detecta o push automaticamente
# Build do Dockerfile em ~60 segundos
# Deploy sem downtime (rolling update)
# Todas as marinas recebem a atualização simultaneamente
```

O que é atualizado automaticamente

Os itens abaixo são gerenciados pelo sistema no startup — **nenhuma intervenção manual é necessária**:

O quê	Mecanismo
Novas tabelas no banco	001_initial.sql rodado por runMigrations() no boot
Tabelas existentes com novas colunas	Arquivo de migração 002_*.sql — aplicado uma vez
Novos dados de configuração	seedTenant() idempotente — INSERT ON CONFLICT DO NOTHING
Novas permissões de perfil	seedDefaultPermissions() — inserção segura
Alterações no frontend	frontend.html substituído — sem rebuild, sem cache
Novo tenant criado após update	provisionTenant() aplica todas as migrações disponíveis

Atualização em VPS — Script Automático

Para servidores VPS, o script `update.sh` realiza todo o processo com segurança em aproximadamente 45 segundos:

1	Verifica	Node.js 20+, git, PM2 e variáveis de ambiente
2	Backup	pg_dump do PostgreSQL com timestamp (retém 30 dias)
3	Fetch	git fetch + exibe commits novos disponíveis
4	Pull	git pull origin main — atualiza código
5	npm install	Instala/atualiza dependências npm (ex: nova versão de postgres.js)
6	Valida	node --check server.js — reverte se houver erro de sintaxe
7	Restart	pm2 restart marina-one --update-env
8	Health	Checa /api/version em loop até confirmar resposta
9	Resumo	Exibe versão anterior → versão nova + hash do commit

```
cd ~/marina-one && bash update.sh
```

Endpoint /api/version

```
{
  "version": "2.0.0",
  "git_hash": "4079f3b",
  "build_date": "2025-04-28",
  "node": "v20.12.0",
  "uptime_sec": 3842,
  "saas": true
}
```

Rollback de Emergência

Atenção: execute o rollback apenas se o sistema apresentar falha crítica após atualização. Para SaaS gerenciado (Railway), basta reverter o deploy na interface web.

```
# VPS – reverter código
pm2 stop marina-one
git log --oneline -5          # identificar hash anterior
git checkout <hash-anterior> # ex: git checkout a1b2c3d
npm install
pm2 restart marina-one

# Se houve migração de banco (ex: nova coluna), reverter schema manualmente
# psql $DATABASE_URL -c "ALTER TABLE marina_porto-belo.users DROP COLUMN nova_coluna;"
```

Política de Versionamento (Semantic Versioning)

Versão	Exemplo	Quando ocorre	Ação necessária
PATCH (x.x.+1)	2.0.1	Bug fix, ajuste visual	Atualização automática segura
MINOR (x.+1.0)	2.1.0	Nova feature, schema additive	Migrações aplicadas no boot
MAJOR (+1.0.0)	3.0.0	Quebra de arquitetura / schema	Instruções específicas na release

Todas as versões são documentadas em `CHANGELOG.md` com data, tipo e descrição.

1

1

Escalabilidade e Roadmap

Limites e Capacidade

Métrica	Valor estimado	Observação
Tenants simultâneos	Ilimitado	Cada schema é isolado — sem interferência
Usuários por tenant	10–50 simultâneos	Limitado pela instância Node.js (single-thread)
Armazenamento por tenant	~50MB para 1000 clientes	PostgreSQL comprime e indexa eficientemente
Conexões DB	Até 10 por tenant (pool)	Configurável em src/db/pool.js
Uptime esperado	>99.9% (Railway/Neon)	SLAs dos provedores gerenciados

Quando escalar horizontalmente

Para carga muito alta (>50 usuários simultâneos por tenant), recomenda-se:

- Aumentar o pool de conexões por tenant em src/db/pool.js
- Adicionar um segundo processo Node.js atrás de um load balancer (NGINX upstream)
- Migrar o PostgreSQL para Neon ou RDS com réplicas de leitura
- Contratar o **plano de personalização** para essas evoluções (R\$ 150/h)

Roadmap Sugerido

Fase	Entregável	Estimativa
Atual	SaaS multi-tenant PostgreSQL — v2.0	Concluído
v2.1	App mobile PWA + notificações push	3–4 semanas
v2.2	Relatórios PDF automáticos por email	2–3 semanas
v2.3	Integração PIX via API (Efí/Gerencianet)	3–5 semanas
v3.0	Marketplace: marinas no mesmo domínio	2–3 meses

1
2

Suporte e Contato

Plano	Valor	Descrição
Implantação SaaS	R\$ 3.000 (único)	Provisionamento completo do ambiente multi-tenant, domínio e
Manutenção Mensal	R\$ 1.000 / mês	Hospedagem, atualizações de todas as marinas e suporte
Nova Marina (tenant)	R\$ 500 (único)	Provisionamento de novo tenant, personalização inicial e onboa
Personalização	R\$ 150 / hora	Desenvolvimento de funcionalidades específicas e integrações

Contato

Arthur Noli
rj.madmax@gmail.com

SaaS Owner · Marina One